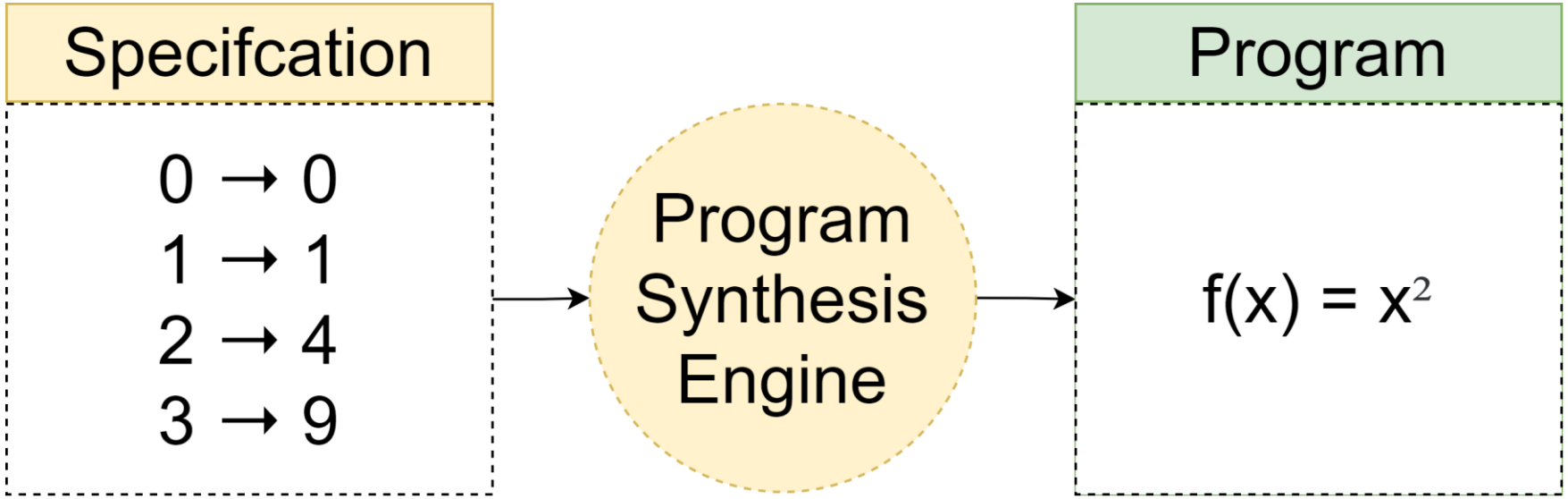
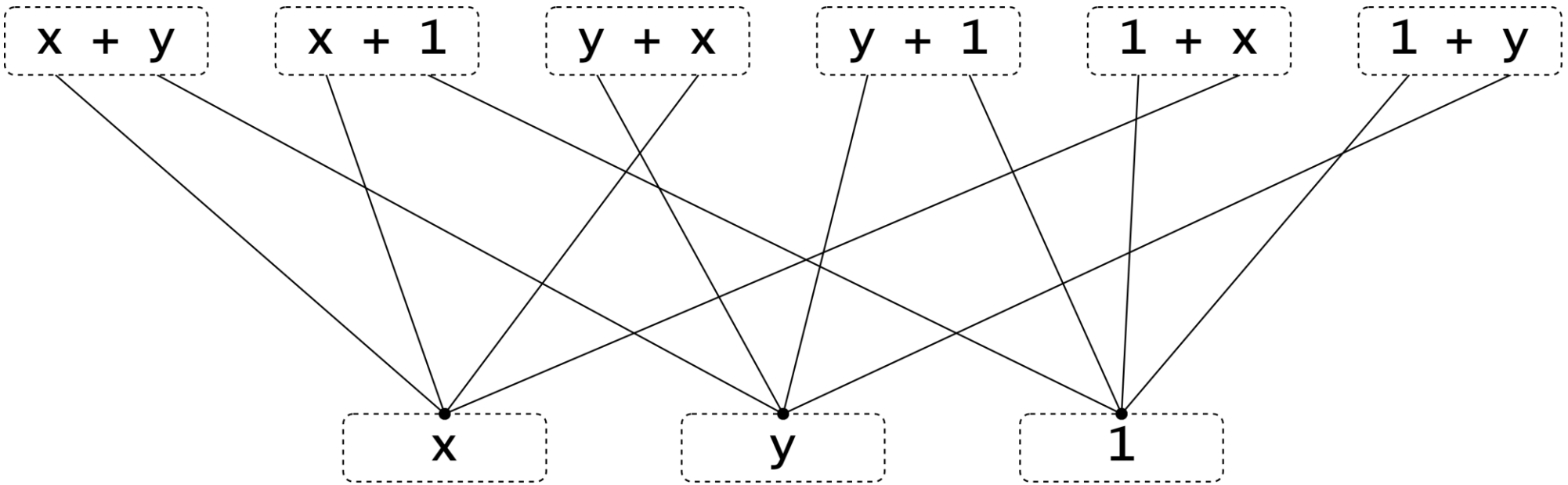
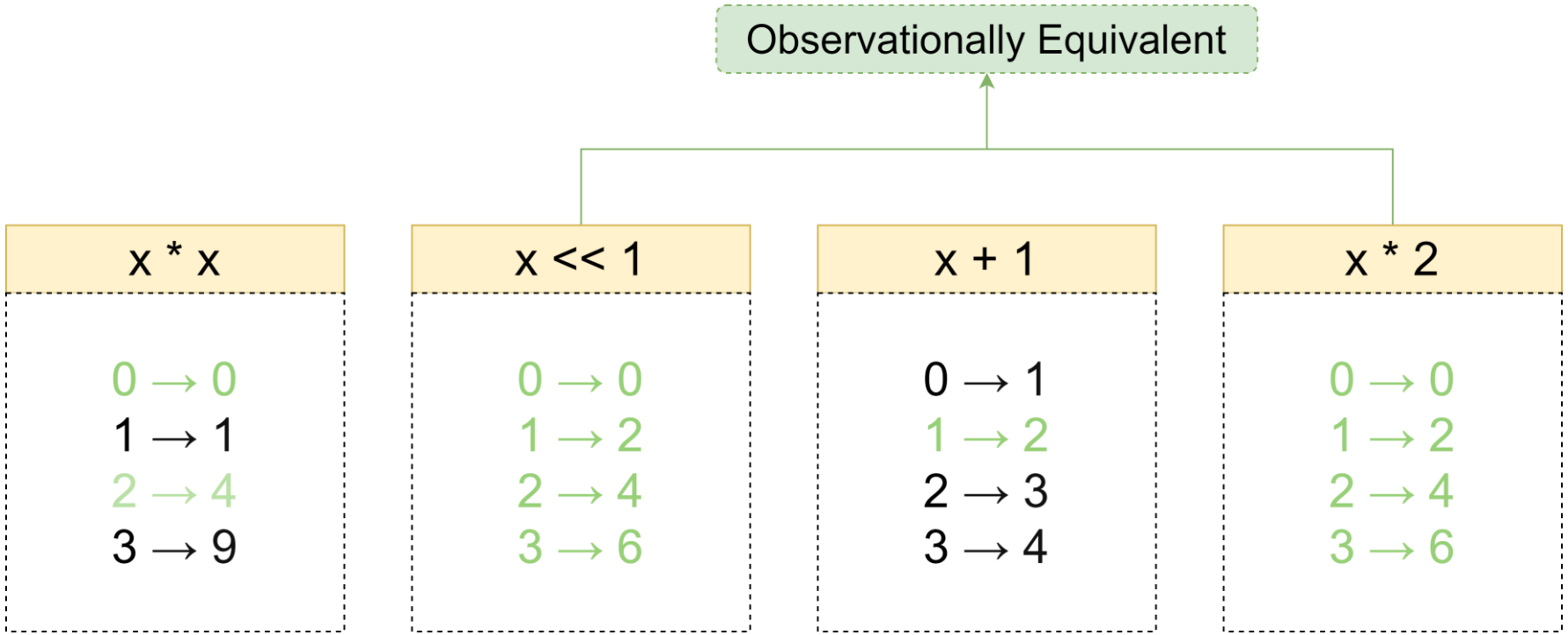
Enumerative Program Synthesis with Observational Equivalence Reduction

Varun Varma

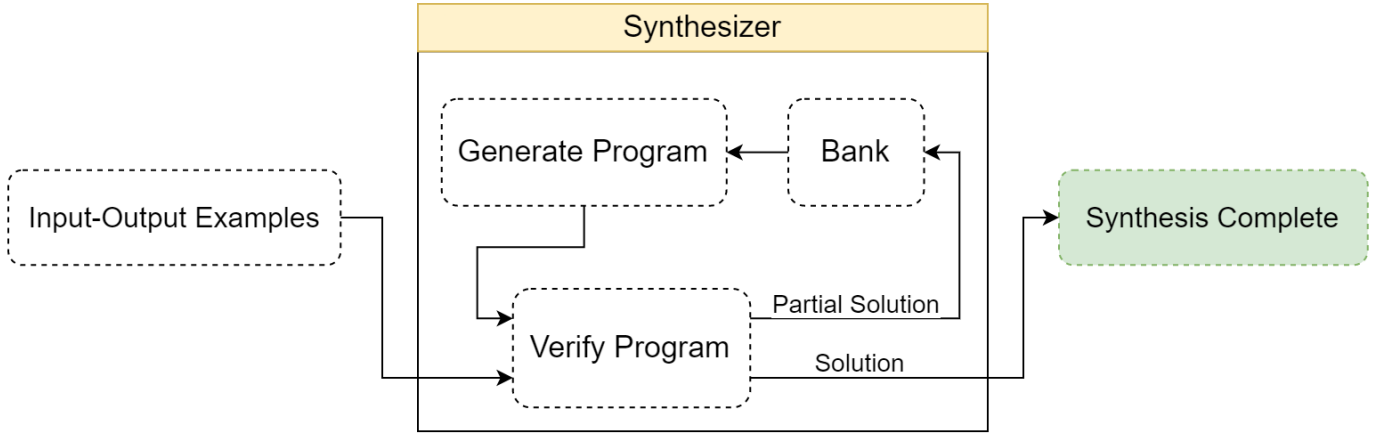

Specifcation
0 → 0
1 → 1
2 → 4
3 → 9
Program Synthesis Engine
Program
f(x) = x²

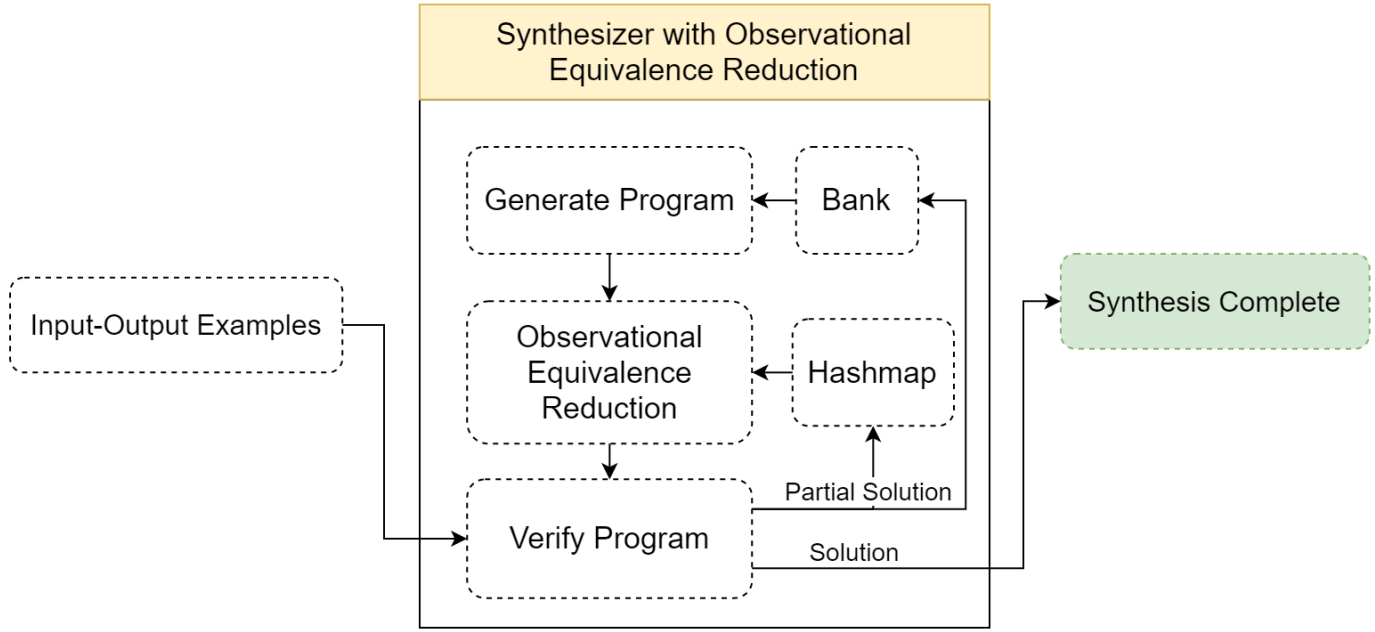

x + y
x + 1
y + x
y + 1
1 + x
1 + y
x
y
1


Observationally Equivalent
x * x
0 → 0
1 → 1
2 → 4
3 → 9
x << 1
0 → 0
1 → 2
2 → 4
3 → 6
x + 1
0 → 1
1 → 2
2 → 3
3 → 4
x * 2
0 → 0
1 → 2
2 → 4
3 → 6

Does observational equivalence reduction optimize enumerative program synthesis?

The enumerative program synthesis engine will find solutions to a set of problems faster with observational equivalence reduction enabled than disabled, because observational equivalence reduction will reduce the size of the search space.

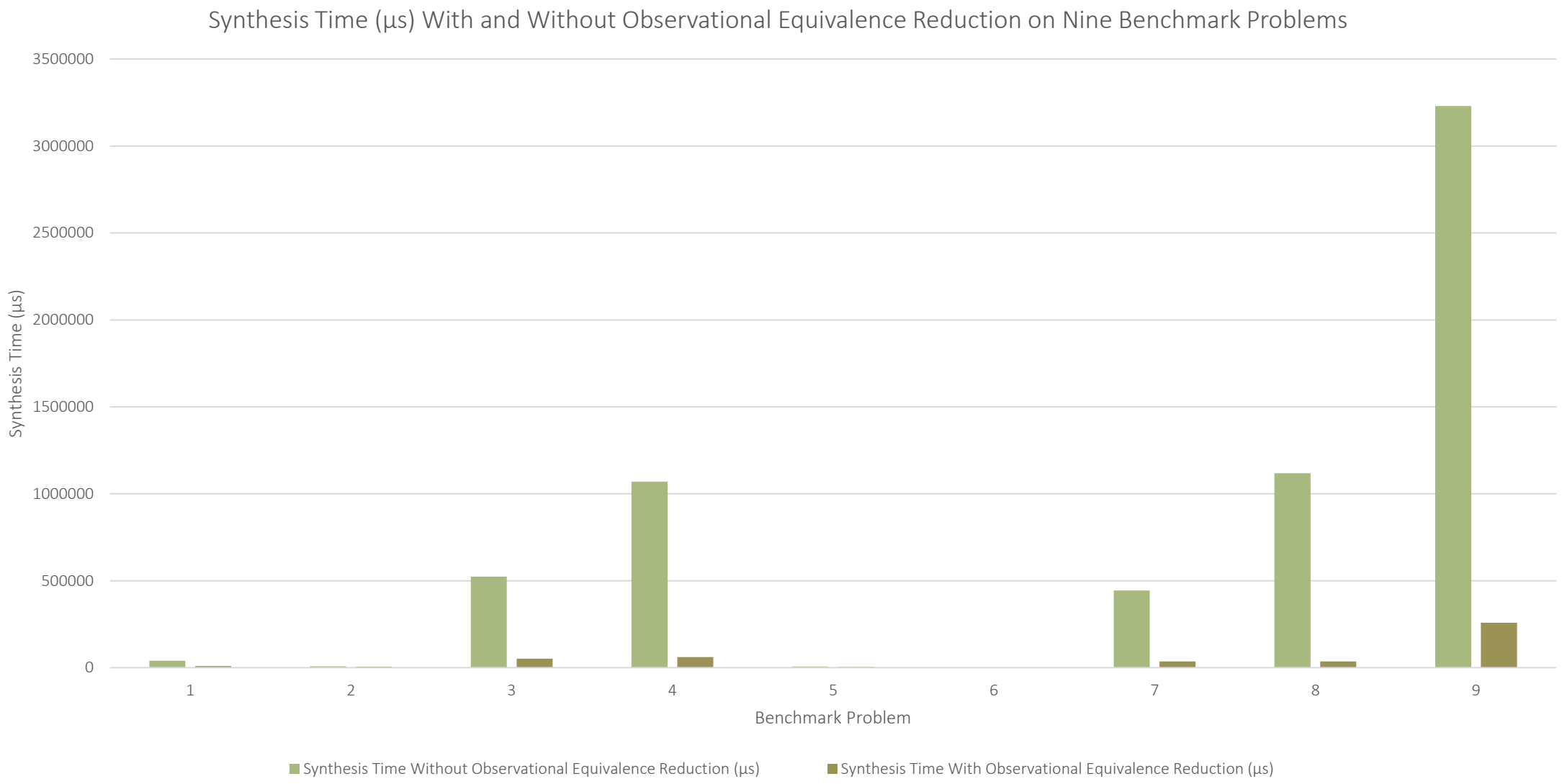

Synthesizer
Generate Program
Bank
Input-Output Examples
Synthesis Complete
Partial Solution
Verify Program
Solution

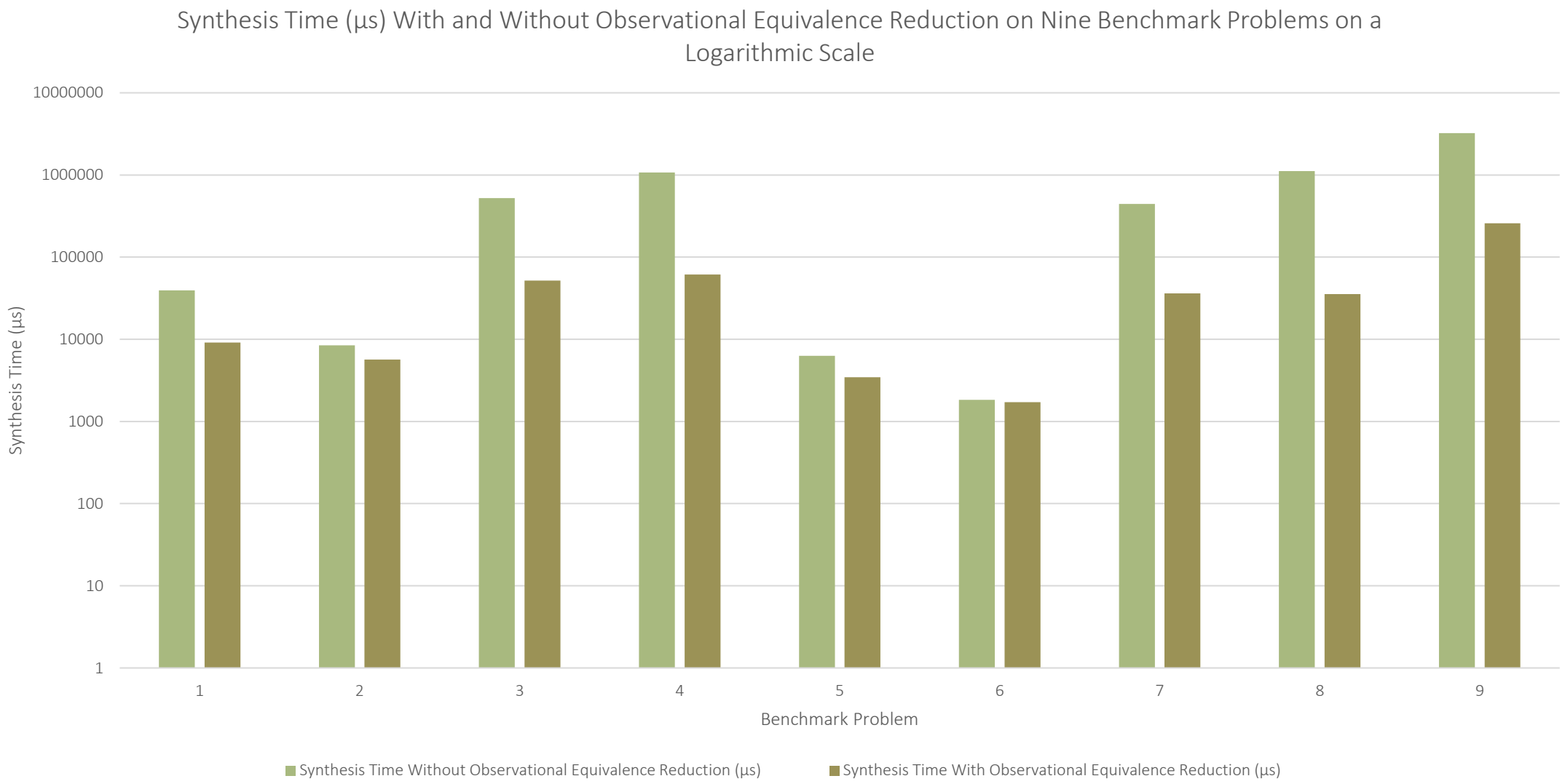

Synthesizer with Observational Equivalence Reduction
Generate Program
Bank
Input-Output Examples
Observational Equivalence Reduction
Hashmap
Synthesis Complete
Partial Solution
Verify Program
Solution

Materials

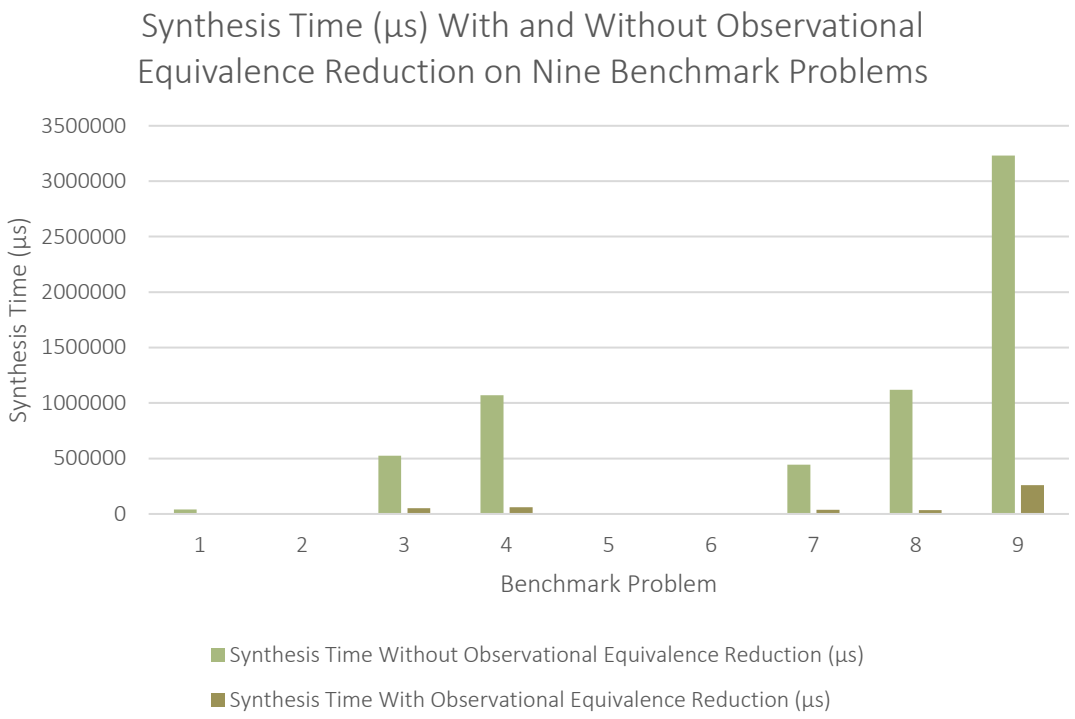
- Rust 1.53 Nightly
- Intel Core i7-9750H processer with 32 GB of RAM on a single thread
- Code can be found at github.com/varranvar/enumerative_synthesis_with_obe

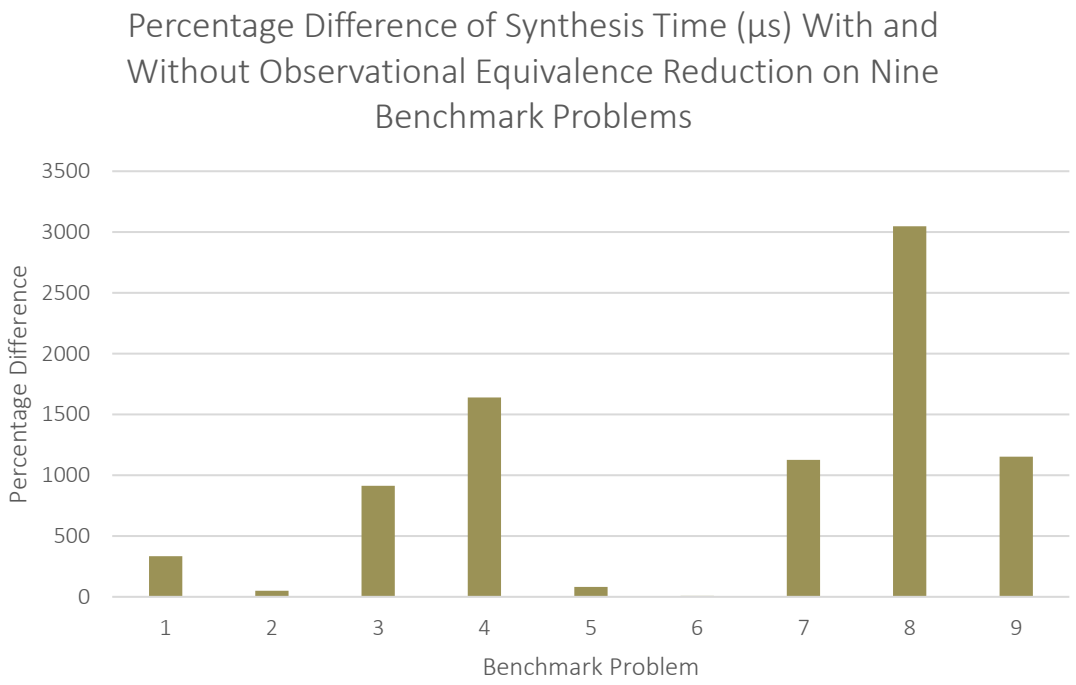
Hacker's Delight Bit Manipulation Programs 1 - 9		
Problem	Description	Program
1	Turn off rightmost bit	1. a = x - 1 2. return x & a
2	Test whether an unsigned integer is of the form 2^{n-1}	1. a = x + 1 2. return x & a
3	Isolate the rightmost bit	1. a = -x 2. return x & a
4	Form a mask that identifies the rightmost bit and the trailing zeros	1. a = x – 1 2. return x ^ a
5	Right propagate rightmost bit	1. a = x – 1 2. return x | a
6	Turn on the right-most zero-bit in a word	1. a = x + 1 2. return x | a
7	Isolate the rightmost zero-bit	1. a = !x 2. b = x + 1 3. return a & b
8	Form a mask that identifies the trailing zeros	1. a = x – 1 2. b = !x 3. return a & b
9	Absolute value function	1. a = x >> max 2. b = x | a 3. return b - a


Synthesis Time (µs) With and Without Observational Equivalence Reduction on Nine Benchmark Problems
3500000
3000000
2500000
2000000
1500000
1000000
500000
0
Synthesis Time (µs)
1
2
3
4
5
6
7
8
9
Benchmark Problem
Synthesis Time Without Observational Equivalence Reduction (µs)
Synthesis Time With Observational Equivalence Reduction (µs)


Synthesis Time (µs) With and Without Observational Equivalence Reduction on Nine Benchmark Problems on a Logarithmic Scale
10000000
1000000
100000
10000
1000
100
10
1
Synthesis Time (µs)
1
2
3
4
5
6
7
8
9
Benchmark Problem
Synthesis Time Without Observational Equivalence Reduction (µs)
Synthesis Time With Observational Equivalence Reduction (µs)

Problem	Synthesized Program Without Observational Equivalence Reduction	Synthesized Program With Observational Equivalence Reduction
1	(& (- x 1) (+ x 0))	(& (- x 1) x)
2	(& (^ x 0) (+ 1 x))	(& x (+ x 1))
3	(& (/ x 0) (- 0 x))	(& x (- 0 x))
4	(^ (- 0 x) (! x))	(^ (+ x -128) (+ 127 x))
5	(| (| 1 x) (- x 1))	(| (| x 1) (- x 1))
6	(| (| 1 x) (+ 1 x))	(| (| x 1) (+ x 1))
7	(& (+ x 1) (! x))	(& (+ 1 x) (! x))
8	(& (- x 1) (! x))	(& (! x) (- x 1))
9	(& (^ (- 0 x) (^ 1 x)) (/ (- 0 x) (>> x 0)))	(& (^ (- 0 x) (^ x 1)) (/ (- 0 x) x))


Synthesis Time (µs) With and Without Observational Equivalence Reduction on Nine Benchmark Problems
Synthesis Time (µs)
3500000
3000000
2500000
2000000
1500000
1000000
500000
0
1
2
3
4
5
6
7
8
9
Benchmark Problem
Synthesis Time Without Observational Equivalence Reduction (µs)
Synthesis Time With Observational Equivalence Reduction (µs)


Percentage Difference of Synthesis Time (µs) With and Without Observational Equivalence Reduction on Nine Benchmark Problems
Percentage Difference
3500
3000
2500
2000
1500
1000
500
0
1
2
3
4
5
6
7
8
9
Benchmark Problem

Conclusions

- The hypothesis was supported
- OER makes enumerative program synthesis more efficient
- OER provides more efficiency gains the harder the problem is
  - Suggests that OER can optimize real world synthesis problems, not just toy examples
- OER can be used to optimize industry-standard program synthesis engines
  - Improvements in program synthesis will accelerate and democratize programming
- Further research could explore using other methods for equivalence reduction, such as e-graphs

Works Cited

Alur, R., Bodik, R., Juniwal, G., Martin, M. M., Raghothaman, M., Seshia, S. A., . . . Udupa, A. (2013). Syntax-guided synthesis. *2013 Formal Methods in Computer-Aided Design*. doi:10.1109/fmcad.2013.6679385

Alur, R., Singh, R., Fisman, D., & Solar-Lezama, A. (2018). Search-based program synthesis. *Communications of the ACM*, *61*(12), 84-93. doi:10.1145/3208071

Barke, S., Peleg, H., & Polikarpova, N. (2020). Just-in-time learning for bottom-up enumerative synthesis. *Proceedings of the ACM on Programming Languages*, *4*(OOPSLA), 1-29. doi:10.1145/3428295

Smith, C., & Albarghouthi, A. (2019). Program synthesis with equivalence reduction. *Lecture Notes in Computer Science*, 24-47. doi:10.1007/978-3-030-11245-5_2

Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). Egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, *5*(POPL), 1-29. doi:10.1145/3434304